Functions

In [ ]:
```
# def function_name(arg1, arg2):

#     statement 1......
#     statement 2......
#     .
#     .
#     .
#     statement n......

#    return statement
```

In [ ]:
```

```

In [1]:
```
def student_info():

    print('Hello Mohit')
```

In [2]:
```
student_info()
```

```
Hello Mohit
```

In [ ]:
```

```

In [3]:
```
def student_info(name):

    print(name)
```

In [6]:
```
student_info('Hello Sourav')
```

```
Hello Sourav
```

In [ ]:
```

```

In [7]:
```
def student_info(name, age, designation):

    print('My name is ' + name + '. I am ' + age + ' year old and I am a ' + designation)
```

In [12]:
```
student_info('Sourav', '26', 'Data Scientist')
```

```
My name is Sourav. I am 26 year old and I am a Data Scientist
```

In [9]:
```
4 + 5
```

Out[9]:
```
9
```

In [10]:
```
'4' + '5'
```

Out[10]:
```
'45'
```

In [11]:
```
'4' + 5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 '4' + 5

TypeError: can only concatenate str (not "int") to str
```

f-string method

In [18]:
```
def student_info(name, age, designation):

    print(f'My name is {name}. I am {age} year old and I am a {designation}')
```

In [19]:
```
student_info('Sourav', '26', 'Data Scientist')
```

```
My name is Sourav. I am 26 year old and I am a Data Scientist
```

In [20]:
```
student_info('Sourav', 26, 'Data Scientist')
```

```
My name is Sourav. I am 26 year old and I am a Data Scientist
```

In [ ]:
```

```

Ways to define a function

- with_argument_without_return
- without_argument_with_return
- with_argument_with_return
- without_argument_without_return

In [21]:
```
def with_argument_without_return(name):

    print(name)
```

In [23]:
```
with_argument_without_return('Himanshu')
```

```
Himanshu
```

In [ ]:
```
```

In [24]:
```
def without_argument_with_return():

    a = 5
    b = 6
    c = a + b

    return c
```

In [28]:
```
a = without_argument_with_return()
a
```

Out[28]: 11

In [29]:
```
without_argument_with_return()
```

Out[29]: 11

In [ ]:
```
```

In [31]:
```
def with_argument_with_return(a, b):

    c =  a + b

    return c
```

In [32]:
```
with_argument_with_return(5, 6)
```

Out[32]: 11

In [ ]:
```
```

In [33]:
```
def without_argument_without_return():

    print('Hello')
```

In [34]:
```
without_argument_without_return()
```

```
Hello
```

In [ ]:
```
```

In [45]:
```
def even_number(data):

    even = []

    for i in data:
        if i%2==0:
            even.append(i)

    return even
```

In [46]:
```
lst = [2,3,4,6,7,3,1,4,8]

even_number(lst)
```

Out[46]:
```
[2, 4, 6, 4, 8]
```

In [47]:
```
even_number([323,2,3,21,4,42,43,21,3,43,55,53,45,45,6])
```

Out[47]:
```
[2, 4, 42, 6]
```

In [ ]:

Types of Function

In [48]:
```
# User-Defined Function
# Built-In Function
# Lambda Function
```

In [49]:
```
def user_define(my_data):

    print(my_data)

user_define('This is a user-defined function')
```

```
This is a user-defined function
```

In [ ]:

Built-In Function

In [50]:
```
lst = [323,2,3,21,4,42,43,21,3,43,55,53,45,45,6]
lst
```

Out[50]:
```
[323, 2, 3, 21, 4, 42, 43, 21, 3, 43, 55, 53, 45, 45, 6]
```

In [51]:
```
len(lst)
```

Out[51]:
```
15
```

In [52]:
```
sum(lst)
```

Out[52]: 709

In [53]:
```
min(lst)
```

Out[53]: 2

In [54]:
```
max(lst)
```

Out[54]: 323

In [55]:
```
lst = [323,2,3,21,4,42,43,21,3,43,55,53,45,45,6]
len(lst)
```

Out[55]: 15

In [60]:
```
def lenn(data):

    counter = 0

    for i in data:
        counter += 1

    return counter
```

In [61]:
```
lst = [323,2,3,21,4,42,43,21,3,43,55,53,45,45,6]
lenn(lst)
```

Out[61]: 15

In [ ]:
```

```

In [68]:
```
lst = [1,3,323,2,3,21,4,42,43,567,21,3,43,55,53,45,45,6]
max(lst)
```

Out[68]: 567

In [65]:
```
def maxx(data):

    largest = data[0]

    for i in data:
        if i > largest:
            largest = i

    return largest
```

In [69]:
```
lst = [1,3,323,2,3,21,4,42,43,567,21,3,43,55,53,45,45,6]
maxx(lst)
```

Out[69]: 567

In [ ]:
```

```

Lambda Function

In [70]:
```
# Without Lambda

def add():

    a = 5
    b = 10

    c = a + b

    return c

add()
```

Out[70]: 15

In [71]:
```
x = lambda a, b: a + b
x(5, 10)
```

Out[71]: 15

In [72]:
```
x = lambda a: a + 10
x(5)
```

Out[72]: 15

In [ ]:
```

```

In [74]:
```
x = lambda z: z + 'How are you?'
x('Hello Sourav, ')
```

Out[74]: 'Hello Sourav, How are you?'

In [ ]:
```

```

Advance Functions

Map()

In [79]:
```
lst = [3,2,4,6,3,2,3,2]

power = map(lambda x: x**2, lst)
power
```

Out[79]:
```
<map at 0x14de3cd4ac0>
```

In [82]:
```
lst = [3,2,4,6,3,2,3,2]

power = list(map(lambda x: x**2, lst))
power
```

Out[82]:
```
[9, 4, 16, 36, 9, 4, 9, 4]
```

In [83]:
```
lst = [3,2,4,6,3,2,3,2]

add = list(map(lambda x: x+10, lst))
add
```

Out[83]:
```
[13, 12, 14, 16, 13, 12, 13, 12]
```

In [ ]:
```

```

Filter()

In [85]:
```
lst2 = list(range(1,21))
lst2
```

Out[85]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

In [87]:
```
even = filter(lambda x: x%2==0, lst2)
even
```

Out[87]:
```
<filter at 0x14de3cd4ee0>
```

In [88]:
```
even = list(filter(lambda x: x%2==0, lst2))
even
```

Out[88]:
```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

In [ ]:
```

```

Reduce()

In [89]:
```
from functools import reduce
```

In [90]:
```
lst3 = [1,2,3,4,5]

add = reduce(lambda x, y: x + y, lst3)
add
```

Out[90]:
```
15
```

In [ ]:
```

```

In [91]:
```
lst3 = [1,2,3,4,5]

add = reduce(lambda x, y: x * y, lst3)
add
```

Out[91]: 120

In [ ]:
```

```

Copy of Functions

In [92]:
```
def my_func():

    return 'my sample function'
```

In [93]:
```
my_func()
```

Out[93]: 'my sample function'

In [94]:
```
x = my_func()
```

In [95]:
```
x
```

Out[95]: 'my sample function'

In [96]:
```
del my_func
```

In [97]:
```
my_func()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[97], line 1
----> 1 my_func()

NameError: name 'my_func' is not defined
```

In [98]:
```
x
```

Out[98]:
```
'my sample function'
```

In [ ]:
```

```

Clousers

In [108]:
```
# Nested Function - Function inside another function

def main_func():
    name = 'Sourav Sharma'

    def local_func():
        print('Local or Inner Function')
        print(name)
        print('Outer or Main function has called!')

    return local_func()
```

In [109]:
```
main_func()
```

```
Local or Inner Function
Sourav Sharma
Outer or Main function has called!
```

In [ ]:

```
```

In [112]:

```
def main_func(name):

    def local_func():
        print('Local or Inner Function')
        print(name)
        print('Outer or Main function has called!')

    return local_func()
```

In [115]:

```
main_func('Sourav Kapil')
```

```
Local or Inner Function
Sourav Kapil
Outer or Main function has called!
```

In [ ]:

```
```

In [126]:

```
def main_func(new_func):
    def local_func():
        print('Local or Inner Function')
        new_func('My List:', [1,2,3,4,5,6])
        print('Outer or Main function has called!')

    return local_func()
```

In [127]:

```
main_func(print)
```

```
Local or Inner Function
My List: [1, 2, 3, 4, 5, 6]
Outer or Main function has called!
```

In [ ]:

```
```

In [134]:

```
def main_func(new_func):
    def local_func():
        print('Local or Inner Function')
        print(new_func([1,2,3,4,5,6]))
        print('Outer or Main function has called!')

    return local_func()
```

In [138]:

```
main_func(sum)
```

```
Local or Inner Function
21
Outer or Main function has called!
```

In [ ]:

```
```

In [146]:

```
def main_func(new_func):
    var = new_func([2,4,2,1,1,5,6,4,5,7])

    def inner_func(x):
        print('This is my inner funtion')
        print('Successfully called main function')
        print(x)


    return inner_func(var)
```

In [147]:

```
main_func(max)
```

```
This is my inner funtion
Successfully called main function
7
```

In [ ]:

Decorators

In [149]:
```
def main_func(new_func):

    def local_func():
        print('Local or Inner Function')
        print('Outer or Main function has called!')

    return local_func()
```

In [150]:
```
def updated_func():
    print('This is my new statement using decorator')
```

In [153]:
```
main_func(updated_func())
```

```
This is my new statement using decorator
Local or Inner Function
Outer or Main function has called!
```

In [155]:
```
a = main_func(updated_func())
```

```
This is my new statement using decorator
Local or Inner Function
Outer or Main function has called!
```

In [ ]:

In [ ]:

In [161]:
```
def main_func(new_func):

    def local_func():
        print('Local or Inner Function')
        print('Outer or Main function has called!')
        new_func()

    return local_func()
```

In [162]:
```
@main_func
def updated_func():
    print('This is my new statement using decorator')
```

```
Local or Inner Function
Outer or Main function has called!
This is my new statement using decorator
```

In [ ]:

In [ ]:

In [168]:
```
def main_func(new_func):

    def local_func():
        print('Local or Inner Function')
        print('Outer or Main function has called!')
        new_func(10)

    return local_func()
```

In [171]:
```
@main_func
def updated_func(x):
    print('This is my new statement using decorator')
    print(x)
```

```
Local or Inner Function
Outer or Main function has called!
This is my new statement using decorator
10
```

In [170]:
```
main_func(updated_func)
```

```
Local or Inner Function
Outer or Main function has called!
This is my new statement using decorator
10
```

In [ ]:
```

```

In [186]:
```
def sourav(func):

    print('This is Sourav Function')
    func()
```

In [187]:
```
@sourav
def shubham():
    print('Shubham and Sourav are the Data Scientist')
```

```
This is Sourav Function
Shubham and Sourav are the Data Scientist
```

In [ ]:
```

```

In [188]:
```
def sourav(func):

    print('This is Sourav Function')
    func()
```

In [189]:
```
def shubham():
    print('Shubham and Sourav are the Data Scientist')
```

In [190]:
```
sourav(shubham)
```

```
This is Sourav Function
Shubham and Sourav are the Data Scientist
```

In [ ]:
```

```

*args and **kwargs

In [201]:
```
def add(a, b, c):

    addd = a + b + c

    return addd
```

In [204]:
```
add(3,4,6)
```

Out[204]:
```
13
```

In [ ]:
```

```

In [207]:
```
def sample(*args):

    a = sum(args)

    return a
```

In [208]:
```
sample(3,4,6)
```

Out[208]: 13

In [ ]:
```

```

In [213]:
```
def sample(*args):

    var = 0
    for i in args:
        var += i

    return var
```

In [214]:
```
sample(3,4,6)
```

Out[214]: 13

In [ ]:
```

```

In [215]:
```
def sample(*sourav):

    a = sourav

    return a
```

In [216]:
```
sample(2,3,4,5,6,7)
```

Out[216]:
```
(2, 3, 4, 5, 6, 7)
```

In [ ]:
```

```

In [ ]:
```
# **kwargs
```

In [221]:
```
def display_info(**kwargs):

    for key, value in kwargs.items():
        print(f'{key} : {value}')
```

In [223]:
```
display_info(name='Sourav', age=26, city='Noida')
```

```
name : Sourav
age : 26
city : Noida
```

In [ ]:
```

```

In [ ]:
```

```